

ARCHITECTURE BLUEPRINT

LMDS

Logistics Master Data System

เอกสารสถาปัตยกรรมระบบ Master Data
สำหรับงานขนส่งและข้อมูลปฏิบัติการ

Version 5.3 — Hybrid Alias Ready

เอกสารฉบับนี้รวบรวมสถาปัตยกรรมระบบ LMDS ตั้งแต่เป้าหมายระบบ โครงสร้างข้อมูล สถาปัตยกรรมแบบแยกชั้น กลไก Pipeline การทำงาน ไปจนถึงแนวทางการป้องกันความเสียหาย เพื่อให้ นักพัฒนาและทีมปฏิบัติการเข้าใจภาพรวมของระบบได้อย่างครบถ้วน

Platform: Google Apps Script + Google Sheets

Core Engine: MatchEngine V5.3

สารบัญ

1

เกี่ยวกับระบบ LMDS

2

อัปเดตล่าสุด — Hybrid Alias Architecture

3

เป้าหมายระบบ

4

โครงสร้างข้อมูลหลัก

4.1 Master Tables

4.2 Transaction / Operations

4.3 System Tables

5

สถาปัตยกรรมแบบแยกชั้น (Layered Architecture)

5.1 Ingestion Layer

5.2 Normalization Layer

5.3 Master Resolution Layer

5.4 Hybrid Alias Layer

5.5 Transaction & Review Layer

5.6 Governance & Hardening Layer

6

โมเดลข้อมูลเชิงตรรกะ (Logical Data Model)

7

The Trinity Framework

8

กระบวนการทำงาน (Execution Flow)

9.1 Phase A: Ingestion

9.2 Phase B: Enrichment & Extraction

9.3 Phase C: Rules Matrix Resolution

9.4 Phase D: Persistence Control

12.1 การติดตั้งครั้งแรก (First-time Setup)

12.2 การใช้งานหลัก

12.3 หมายเหตุสำคัญก่อน Production Run

1. เกี่ยวกับระบบ LMDS

LMDS (Logistics Master Data System) คือระบบ Master Data สำหรับงานขนส่งที่ทำหน้าที่รับข้อมูลดิบจากงานประจำวัน ทำความสะอาดข้อมูล (Data Cleansing) จับคู่กับฐาน Master ที่มีอยู่แล้ว (Master Matching) และบันทึกผลเชิงธุรกรรมลงตาราง `FACT_DELIVERY` เพื่อให้ทีมปฏิบัติการสามารถใช้งานข้อมูลได้อย่างต่อเนื่องและตรวจสอบย้อนหลังได้ ระบบถูกพัฒนาบนแพลตฟอร์ม Google Apps Script ควบคู่กับ Google Sheets ซึ่งช่วยให้สามารถเข้าถึงข้อมูลได้ง่าย แก้ไขได้ทันที และทำงานร่วมกับทีมปฏิบัติการได้อย่างราบรื่น

จุดเด่นสำคัญของ LMDS คือการเป็นทั้ง Master Data Repository และ Matching Engine ในระบบเดียวกัน โดยระบบออกแบบมาเพื่อรับมือกับข้อมูลขนส่งที่คุณภาพไม่สม่ำเสมอ อาจมีการพิมพ์ผิด ชื่อไม่ตรงกัน ที่อยู่ไม่ครบ หรือข้อมูลซ้ำซ้อน ระบบจะทำการ Normalize ข้อมูลเหล่านั้น จับคู่กับ Master ที่มีอยู่ และตัดสินใจว่าจะสร้างรายการใหม่ จับคู่อัตโนมัติ หรือส่งเข้าคิวตรวจสอบโดยมนุษย์ (Human-in-the-loop) ตามความเหมาะสม

2. อัปเดตล่าสุด — Hybrid Alias Architecture

ในเวอร์ชัน V5.3 ระบบได้เพิ่มแนวทาง Hybrid Alias Architecture ซึ่งเป็นการขยายความสามารถในการจับคู่ข้อมูล (Matching) ให้มีประสิทธิภาพมากยิ่งขึ้น โดยเฉพาะในกรณีที่ชื่อหรือข้อมูลพิมพ์ไม่ตรงมาตรฐาน ซึ่งเคยเป็นสาเหตุหลักของ False Negative ในระบบเดิม

จุดเปลี่ยนสำคัญของ Hybrid Alias

- เพิ่มตารางกลาง `M_ALIAS` สำหรับจัดการ alias ข้ามโดเมน (Cross-domain Alias Ledger) ทำให้สามารถค้นหา alias ระดับ Global ได้โดยตรงโดยไม่ต้องผ่านตารางย่อยของแต่ละ Entity
- รองรับ `master_uuid` ใน `M_PERSON` และ `M_PLACE` ซึ่งเป็นตัวเชื่อมระหว่าง Entity เดียวกันที่อาจมีหลายรูปแบบชื่อหรือหลายรายการในตาราง Master
- เชื่อม Fast-path สำหรับค้นหา alias ระดับ Global ใน Person/Place Candidate Matching ช่วยลดเวลาในการประมวลผลและลดอัตรา False Negative อย่างมีนัยสำคัญ

สิ่งสำคัญคือระบบยังคงความเข้ากันได้กับ alias เดิม (`M_PERSON_ALIAS` , `M_PLACE_ALIAS`) โดยสมบูรณ์ ทำให้การย้ายระบบ (Migration) สามารถทำได้ทีละส่วนโดยไม่กระทบการทำงานปัจจุบัน นอกจากนี้ ระบบโพส์ตที่การใช้งานกับข้อมูลใหม่ได้ทันที โดยไม่บังคับให้ต้อง Backfill ข้อมูลเก่า ตามนโยบายที่ตกลงกันในรอบปรับปรุงนี้

3. เป้าหมายระบบ

LMDS ออกแบบเพื่อเป็น Master Data + Matching Engine สำหรับข้อมูลขนส่งที่คุณภาพไม่สม่ำเสมอ โดยเน้น 3 เสาหลัก:



ทั้งสามเสาหลักนี้ทำงานร่วมกันเพื่อให้มั่นใจว่าข้อมูลที่ผ่านระบบ LMDS จะมีคุณภาพสูง สามารถตรวจสอบย้อนหลังได้ทุกขั้นตอน และทีมปฏิบัติการสามารถใช้งานได้อย่างต่อเนื่องโดยไม่มีการขาดช่วง Data Quality หมายถึงการที่ข้อมูลถูก Normalize ทำความสะอาด และตรวจสอบความถูกต้องก่อนเข้าสู่ระบบ Traceability หมายถึงทุกการตัดสินใจของระบบมีหลักฐานบันทึกไว้สำหรับการตรวจสอบภายหลัง และ Operational Continuity หมายถึงระบบสามารถรันได้ทันทีกับข้อมูลใหม่โดยไม่ต้อง Backfill ข้อมูลเก่า

4. โครงสร้างข้อมูลหลัก

4.1 Master Tables

ตาราง Master เป็นฐานข้อมูลหลักของระบบ LMDS ที่เก็บข้อมูลมาตรฐานสำหรับบุคคล สถานที่ และจุดพิกัด ซึ่งจะถูกใช้เป็นตัวอ้างอิงในการจับคู่ข้อมูลดิบที่เข้ามาในระบบ ข้อมูลในตารางเหล่านี้จะถูกสร้างและปรับปรุงโดย Match Engine โดยอัตโนมัติ หรือผ่านการตรวจสอบโดยมนุษย์ในกรณีที่ระบบไม่สามารถตัดสินใจได้ด้วยตนเอง

ตารางที่ 1: รายการ Master Tables

ตาราง	คำอธิบาย
M_PERSON	ข้อมูลบุคคลหลัก ประกอบด้วย ชื่อ เบอร์โทร หมายเหตุ และ master_uuid สำหรับเชื่อมโยงข้ามโดเมน
M_PERSON_ALIAS	Alias ระดับ Local สำหรับบุคคล เก็บชื่อหรือรูปแบบอื่นที่ใช้เรียกบุคคลเดียวกัน
M_PLACE	ข้อมูลสถานที่หลัก ประกอบด้วย ที่อยู่ ชื่อสถานที่ และ master_uuid สำหรับเชื่อมโยงข้ามโดเมน
M_PLACE_ALIAS	Alias ระดับ Local สำหรับสถานที่ เก็บชื่อหรือรูปแบบอื่นที่ใช้เรียกสถานที่เดียวกัน
M_ALIAS	Global Alias Ledger ตารางกลางสำหรับจัดการ alias ข้ามโดเมน รองรับ entity_type , confidence , source
M_GEO_POINT	จุดพิกัด GPS ที่คนขับเช็คอิน พร้อมรัศมี GEO_RADIUS_M สำหรับการจับรัศมีขยะ (Duplicate Location Merging)
M_DESTINATION	ตารางศูนย์กลางสร้าง Intersection Object Map (Person_ID + Place_ID + Geo_ID = 1 Destination Node)

4.2 Transaction / Operations

ตารางธุรกรรมและปฏิบัติการเป็นที่เก็บข้อมูลการทำงานจริงของระบบ ตั้งแต่ข้อมูลดิบที่รับเข้ามา ไปจนถึงผลลัพธ์ที่ผ่านการประมวลผลแล้ว ข้อมูลในส่วนนี้มีการเปลี่ยนแปลงตลอดเวลาตามการทำงานของ Pipeline

ตารางที่ 2: รายการ Transaction / Operations Tables

ตาราง	คำอธิบาย
FACT_DELIVERY	ตารางธุรกรรมหลัก เก็บผลลัพธ์การจัดส่งที่ผ่านการประมวลผลแล้ว ผูกกับ person_id , place_id , geo_id , dest_id
Q_REVIEW	คิวรอตรวจสอบสำหรับเคสคำถามที่ระบบไม่สามารถตัดสินใจอัตโนมัติได้ รอการตัดสินใจจากมนุษย์
ตารางงานประจำวัน	ข้อมูลงานรายวันที่ดึงจาก SCG API เป็นแหล่งข้อมูลดิบหลักของระบบ
SCGนครหลวงJWDภูมิภาค	Landing Sheet สำหรับข้อมูลดิบจาก SCG API ก่อนเข้าสู่กระบวนการประมวลผล

4.3 System Tables

ตารางระบบเป็นส่วนที่ควบคุมการทำงานภายในของ LMDS ตั้งแต่การตั้งค่าระบบ การบันทึกประวัติ ไปจนถึงข้อมูลภูมิศาสตร์ของประเทศไทยที่ใช้ในการ Normalize ที่อยู่

ตารางที่ 3: รายการ System Tables

ตาราง	คำอธิบาย
SYS_CONFIG	ตารางตั้งค่าระบบ เก็บ Configuration ต่างๆ เช่น API Key, พารามิเตอร์การทำงาน
SYS_LOG	ตารางบันทึกประวัติการทำงานของระบบ (Audit Log) สำหรับการตรวจสอบย้อนหลัง
SYS_TH_GEO	ฐานข้อมูลภูมิศาสตร์ไทย ประกอบด้วย จังหวัด อำเภอ ตำบล รหัสไปรษณีย์ ประมาณ 7,537 รายการ
MAPS_CACHE	แคชผลลัพธ์จาก Google Maps API เพื่อลดการเรียก API ซ้ำและประหยัด Quota

5. สถาปัตยกรรมแบบแยกชั้น (Layered Architecture)

LMDS ออกแบบด้วยสถาปัตยกรรมแบบแยกชั้น (Layered Architecture) ที่แบ่งการทำงานออกเป็น 6 ชั้นหลัก แต่ละชั้นมีหน้าที่เฉพาะและทำงานร่วมกันผ่านกระบวนการ Pipeline ที่ชัดเจน การแบ่งชั้นแบบนี้ช่วยให้สามารถปรับปรุงหรือแก้ไขส่วนใดส่วนหนึ่งได้โดยไม่กระทบส่วนอื่น และช่วยให้การตรวจสอบปัญหา (Debugging) ทำได้ง่ายขึ้น เนื่องจากข้อมูลจะไหลผ่านแต่ละชั้นตามลำดับ

Layer A Ingestion Layer

ชั้นรับข้อมูลดิบเข้าสู่ระบบ เป็นจุดเริ่มต้นของ Pipeline ทั้งหมด ข้อมูลจะถูกดึงจากแหล่งต่างๆ และวางลงใน Landing Sheets เพื่อเตรียมเข้าสู่กระบวนการประมวลผลในชั้นถัดไป

- แหล่งข้อมูล: SCG API, ไฟล์รายวัน, Input จากผู้ใช้งาน
- Landing Sheets: SCGนครหลวงJWDภูมิภาค , ตารางงานประจำวัน , Input

Layer B Normalization Layer

ชั้นทำความสะอาดข้อมูลและปรับรูปแบบให้เป็นมาตรฐานเดียวกัน ข้อมูลดิบที่เข้ามามักมีรูปแบบไม่สม่ำเสมอ ชั้นนี้จะทำให้ข้อมูลพร้อมสำหรับการจับคู่ในชั้นถัดไป

- โมดูล: `05_NormalizeService.gs` , `20_ThGeoService.gs`
- งานหลัก: ทำความสะอาดชื่อ, เบอร์โทร, ที่อยู่, จังหวัด/อำเภอ/ตำบล, รหัสไปรษณีย์

Layer C Master Resolution Layer

ชั้นจับคู่และตัดสินใจ ข้อมูลที่ผ่านการ Normalize แล้วจะถูกนำมาจับคู่กับฐานข้อมูล Master ที่มีอยู่ แต่ละประเภทข้อมูลมี Service จัดการเฉพาะ และ Match Engine จะเป็นผู้ตัดสินใจสุดท้าย

- Person: `06_PersonService.gs`
- Place: `07_PlaceService.gs`
- Geo: `08_GeoService.gs`
- Destination: `09_DestinationService.gs`
- Core Decision: `10_MatchEngine.gs`

Layer D Hybrid Alias Layer (ใหม่)

ชั้นจัดการ Alias แบบ Hybrid ที่รองรับทั้ง Local Alias (เดิม) และ Global Alias Ledger (ใหม่) ชั้นนี้ช่วยลดปัญหา False Negative ในเคสที่ชื่อหรือข้อมูลไม่ตรงมาตรฐาน โดยเชื่อมโยง Entity เดียวกันที่มีหลายรูปแบบผ่าน `master_uuid`

- Local Alias: `M_PERSON_ALIAS` , `M_PLACE_ALIAS`
- Global Alias Ledger: `M_ALIAS`
- Cross-domain Identity: `master_uuid` ใน `M_PERSON` , `M_PLACE`
- Runtime Fast-path: resolve global alias → `master_uuid` → map → `person_id` / `place_id`

Layer E Transaction & Review Layer

ชั้นบันทึกผลลัพธ์และจัดการเคสที่ต้องตรวจสอบ ข้อมูลที่ผ่านการจับคู่แล้วจะถูกบันทึกลง `FACT_DELIVERY` ส่วนเคสที่กำกวมจะถูกส่งเข้าคิว `Q_REVIEW` เพื่อรอการตัดสินใจจากมนุษย์

- ธุรกรรม: ข้อมูลลง `FACT_DELIVERY`
- คิวรอตรวจ: เคสคลุมเครือเข้า `Q_REVIEW`
- Human Decision: CREATE_NEW / MERGE / IGNORE / ESCALATE

Layer F Governance & Hardening Layer

ชั้นควบคุมคุณภาพและความมั่นคงของระบบ ประกอบด้วย การตรวจสอบก่อนรัน (Preflight Checks) การบันทึกประวัติ (Audit) การตรวจสอบคุณภาพ (Quality Reporting) และการจัดการ Review

- โมดูล: `19_Hardening.gs` , `12_ReviewService.gs` , `13_ReportService.gs`
- งานหลัก: Preflight checks, Audit, Log, Quality reporting

6. โมเดลข้อมูลเชิงตรรกะ (Logical Data Model)

โมเดลข้อมูลเชิงตรรกะของ LMDS ออกแบบมาเพื่อรองรับความสัมพันธ์ระหว่าง Entity หลักทั้ง 4 ประเภท ได้แก่ บุคคล สถานที่ จุดพิกัด และ Alias โดยมี `master_uuid` เป็นกุญแจหลักในการเชื่อมโยงข้ามโดเมน และ `FACT_DELIVERY` เป็นตารางศูนย์กลางที่เก็บผลลัพธ์ธุรกรรมทั้งหมด

ตารางที่ 4: Logical Data Model

Entity	คอลัมน์สำคัญ	คำอธิบาย
M_PERSON	<code>person_id</code> , <code>master_uuid</code> , ...	ข้อมูลบุคคลหลัก เชื่อมโยงด้วย <code>master_uuid</code>
M_PLACE	<code>place_id</code> , <code>master_uuid</code> , ...	ข้อมูลสถานที่หลัก เชื่อมโยงด้วย <code>master_uuid</code>
M_ALIAS	<code>alias_id</code> , <code>master_uuid</code> , <code>variant_name</code> , <code>entity_type</code> , <code>confidence</code> , <code>source</code> , <code>created_at</code> , <code>active_flag</code>	Global Alias Ledger สำหรับข้ามโดเมน

Entity	คอลัมน์สำคัญ	คำอธิบาย
FACT_DELIVERY	person_id , place_id , geo_id , dest_id , ...	ตารางธุรกรรมหลัก ผูกกับทุก Entity

ความสัมพันธ์หลัก

M_PERSON และ M_PLACE ใช้ master_uuid เชื่อมโยงกับ M_ALIAS ซึ่งเก็บ Variant Name ทั้งหมดของ Entity นั้นๆ ส่วน FACT_DELIVERY อ้างอิงกลับไปยัง person_id , place_id , geo_id , dest_id เพื่อสร้างความสมบูรณ์ของข้อมูลธุรกรรม

7. The Trinity Framework

แกนหลักของ LMDS Architecture รันด้วยตรรกะแบบแยกฐานข้อมูลเชิงสัมพันธ์ที่เรียกว่า "The Trinity Framework" โดยหลักการคือ การมีอยู่ของการจัดส่ง (Transaction/Fact) 1 ชิ้น จะผูกกันด้วย 3 เสาหลัก บวกกับตารางศูนย์กลางอีก 1 ตาราง ดังนี้



M_PERSON (WHO): ระบบทำการกรอง Phone และ Note (Alias List, Company suffix) จากข้อมูล Unstructured เพื่อ Identify บุคคล ข้อมูลที่เข้ามามักมีเบอร์โทร รหัส หรือหมายเหตุปนอยู่ในฟิลด์ชื่อ ซึ่ง Normalizer จะทำการแยกและจัดเก็บให้เหมาะสม

M_PLACE (WHERE-Address): นำชื่อจากระบบ SCG (`RAW_ADDRESS`) รวมกับ `RESOLVED_ADDR` มาประกอบร่าง และสวมชุดข้อมูลด้วยฐานข้อมูล ปณ. ท้องถิ่น `SYS_TH_GEO` เพื่อให้ได้ที่อยู่ที่มีสมบูรณ์และถูกต้องตามมาตรฐาน

M_GEO_POINT (WHERE-Coordinate): แยก Coordinate ที่คนขับเช็คอิน และเก็บค่าระยะเพื่อ `GEO_RADIUS_M` เพื่อทำการ "จับรัศมีขยะ" (Duplicate Location Merging) โดยจุดที่อยู่ในรัศมี 50 เมตร จะถูกจัดเป็นจุดเดียวกัน

M_DESTINATION: ตารางศูนย์กลางเพื่อสร้าง Intersection Object Map โดย `Person_ID + Place_ID + Geo_ID = 1 Destination Node` ทำให้สามารถอ้างอิงการจัดส่งได้อย่างชัดเจนและสมบูรณ์

8. กระบวนการทำงาน (Execution Flow)

กระบวนการทำงานหลักของ LMDS (Happy Path) ดำเนินไปตามลำดับ 6 ขั้นตอน ตั้งแต่การรับข้อมูลดิบเข้าสู่ระบบ จนถึงการเขียนผลลัพธ์ลงฐานข้อมูลและ Auto-enrich Alias แต่ละขั้นตอนมีรายละเอียดดังนี้



ขั้นตอนที่ 1 - รับข้อมูลดิบ: ระบบดึงข้อมูลงานรายวันจาก SCG API และแหล่งอื่นๆ ลง Landing Sheets ข้อมูลที่เข้ามาจะถูกกรองเฉพาะ Record ที่ `SYNC_STATUS` ยังไม่เป็น SUCCESS

ขั้นตอนที่ 2 - Normalize: ข้อมูลดิบจะถูกทำความสะอาด ทั้งชื่อ เบอร์โทร ที่อยู่ จังหวัด/อำเภอ/ตำบล และรหัสไปรษณีย์ ข้อมูลที่ไม่ตรงมาตรฐานจะถูกปรับให้อยู่ในรูปแบบเดียวกัน

ขั้นตอนที่ 3 - Resolve: ข้อมูลที่ผ่านการ Normalize แล้วจะถูกนำไปจับคู่กับ Master ที่มีอยู่ ทั้ง Person, Place และ Geo โดยแต่ละ Service จะทำการค้นหา Candidate ที่ใกล้เคียงที่สุด

ขั้นตอนที่ 4 - Match Engine Decision: Match Engine จะประเมินผลจากกฎทั้ง 8 ข้อ (Rules Matrix) เพื่อตัดสินใจว่าจะ CREATE_NEW, AUTO_MATCH หรือส่งเข้า REVIEW ตามคะแนน Confidence ที่คำนวณได้

ขั้นตอนที่ 5 - Persistence: ผลลัพธ์จะถูกเขียนลง FACT_DELIVERY หากตรงกัน หรือส่งเข้า Q_REVIEW หากยังกำกวม การเขียนข้อมูลใช้เทคนิค Batch + RangeList เพื่อลด Overhead ของ API

ขั้นตอนที่ 6 - Auto-enrich Alias: ระบบจะนำข้อมูลจริงที่ผ่านการยืนยันแล้วมาเพิ่ม Alias ใหม่เข้าสู่ระบบโดยอัตโนมัติ ทำให้การจับคู่ในรอบถัดไปมีประสิทธิภาพมากขึ้น

9. Global Pipeline Mechanics

MatchEngine ทำงานบนระบบ Asynchronous Concept ของ Google Apps Script โดยแบ่งการทำงานออกเป็น 4 Phase หลัก ซึ่งแต่ละ Phase มีกลไกการทำงานที่ซับซ้อนและมีการจัดการทรัพยากรอย่างรัดกุมเพื่อรองรับข้อจำกัดของแพลตฟอร์ม GAS

9.1 Phase A: Ingestion (SourceRepository.gs)

Phase แรกเป็นการอ่านข้อมูลดิบจาก Landing Sheets โดยมีกลไกสำคัญคือการจำกัดเพดาน Caching เฉพาะ Record ที่ `SYNC_STATUS ≠ SUCCESS` เท่านั้น ซึ่งช่วยลดปริมาณข้อมูลที่ต้องประมวลผลในแต่ละรอบ หลังจากนั้นระบบจะทำการ Filter และสร้าง Object Context สำหรับแต่ละ Record โดยรวบรวมข้อมูล `sysAddr` และ `rawPlaceName` เพื่อส่งต่อไปยัง Phase ถัดไป

9.2 Phase B: Enrichment & Extraction (NormalizeService.gs / PlaceService.gs)

Phase ที่สองเป็นการเติมแต่งและสกัดข้อมูล โดยนำ Name เข้า Normalizer เพื่อสกัดเลขรหัส (`\b[0-9]{8,}\b`) หรือเบอร์โทร (`+66 ..`) ข้อมูลที่เป็น "ขยะ" จะไม่ถูกทิ้ง แต่จะถูกนำไป Push Array แยกด้วย Comma ลงคอลัมน์ `NOTE` เพื่อเก็บไว้เป็น Context สำหรับ Deep Note Search ในภายหลัง

สำหรับ Geo Hierarchy Strategy ระบบจะนำ Text มาแกะด้วย Array Regex หรือโค้ดไปรษณีย์ แล้วเข้า Dictionary ในกรณี Fallback (เป็น Plus Code + ภูมิลำเนาหว่ง) สคริปต์ `lookupPlaceAdminById()` ในไฟล์ 07 จะวิ่งเข้าคู่คืน Province & District กลับไปช่วย `GeoService.gs`

9.3 Phase C: Rules Matrix Resolution (10_MatchEngine.gs)

Phase ที่สามเป็นหัวใจของระบบ โดย MatchEngine จะประเมินตามตารางน้ำหนัก 8 กฎหลัก เพื่อตัดสินใจ Action ที่เหมาะสมสำหรับแต่ละ Record กฎที่สำคัญมีดังนี้

RULE 1

พิกัดหาย: พิกัดจาก Source หายไป = ตรวจจับเป็น `REVIEW_INVALID` Confidence: 0 ทันที
กฎนี้เป็นกฎล็อกที่สำคัญที่สุด เพราะข้อมูลที่ไม่มีพิกัดไม่สามารถนำไปใช้งานได้

RULE 3.5

Tiered Spatial: เช็ครัศมีระหว่างจุดปัจจุบันกับจุดใน Master — ถ้า < 50m ยอมให้
AutoMerge / 50-79.9m ขึ้น Review Warning (สีเหลือง) / 80-100m ขึ้น Warning อ่อน (สี
ส้ม) / > 100m ถือเป็น Area ใหม่หมด

RULE 4-7

Scoring: การ Scoring อิงจากการประเมินของ *Dice Coefficient* กับ *Levenshtein Distance*
โดยคะแนน > 90 = Auto Match / < 50 = Not Found การตัดสินใจออกเป็น Object State
`decision.action = CREATE_NEW / AUTO_MATCH / REVIEW`

9.4 Phase D: Persistence Control (Checkpoint & Chunking)

Phase สุดท้ายเป็นการจัดการการเขียนข้อมูลลงฐานข้อมูล โดยมีกลไกสำคัญ 2 ประการ

Batch Writing: จัด Data เข้า Array ทูกรอบ Batches โดยสาด Record Status ผ่าน `RangeList` ซึ่งใช้ตัว
A1 Notations คอนเวิร์ท Column+Row ลงกระดาน Excel ช่วยลด Overhead API Data Write ไปได้มากถึง
15-25% ของ Execution Time

Execution Time Guard: สคริปต์จะนับ `Date.now()` หากเกิน 300,000 มิลลิวินาที (5 นาที) จะเซต
Trigger Script สวมวิญญาณ Job ยิงคำสั่งตื่นภายใน 60 วินาที และตัวเอง Kill การทำงาน เพื่อป้องกันอาการ
ค้างหรือ Corrupted Caches ซึ่งเป็นข้อจำกัดที่สำคัญของ Google Apps Script ที่มีเวลา Execution จำกัด

10. Dependencies Matrix

ระบบ LMDS มี Dependencies ที่สำคัญ 2 ส่วนหลัก ซึ่งมีผลต่อประสิทธิภาพและความเสถียรของระบบโดยรวม
การจัดการ Dependencies เหล่านี้เหมาะสมเป็นปัจจัยสำคัญในการทำให้ระบบสามารถทำงานได้อย่างต่อ
เนื่องและเสถียร

1. Global Geo Dictionary Cache

`_GLOBAL_GEO_DICT_CACHE` (Global Object Array) ป้องกัน I/O DataRead Timeout สำหรับข้อมูล
ประมวลระดับ 7,537 rows ใน `SYS_TH_GEO` + 16 Array Data Models (Postcode + Note types) โดย
การโหลดข้อมูลเข้า Memory ครั้งเดียวตอนเริ่มระบบ ช่วยลดการอ่าน Sheet ซ้ำซึ่งเป็น Bottleneck หลักของ
GAS

2. Google Maps API + Hybrid Cache

ใช้บริการ `Maps.newDirectionFinder` ซึ่ง Google อนุญาตให้ใช้งานในบริบท Scripts ได้ โดยเขียน Hybrid Cache ที่ผสมผสานระหว่าง Memory Cache (6 ชม.) + Physical Google Sheet Logs (Forever) เพื่อป้องกัน Rate Limits และลดต้นทุนการเรียก API

11. การป้องกันความเสียหายและความถูกต้อง (Disaster & Integrity Prevention)

ระบบ LMDS มีกลไก Audit Traits ผังอยู่ใน `19_Hardening.gs` และ Evidence Trace เพื่อรับประกันความถูกต้องและสามารถตรวจสอบย้อนหลังได้ทุกกรณี ข้อมูลมีระดับของ Security และ Integrity ที่สำคัญ 2 ประการดังนี้

1. Error Handling & Status Reporting

หาก SCG API หรือ Pipeline เขียนล้มเหลว คอลัมน์ `SYNC_STATUS` ต้อง Report = "ERROR" + Background แดง เพื่อให้ทีมปฏิบัติการเห็นได้ทันทีว่า Record ใดมีปัญหา และสามารถติดตามแก้ไขได้อย่างรวดเร็ว

2. Evidence Trace & Backtrace

Record ลงฐาน `FACT_DELIVERY` และตาราง Master ทราบเสมอว่ารหัส ID ขึ้นนั้นมาได้ด้วยอัลกอริทึมใด เนื่องจากติดตาม `match_evidence` (เช่น `name|geo`) ไว้ตรวจสอบ Backtrace ภายหลัง ทำให้ทุกการตัดสินใจของระบบมีหลักฐานที่ตรวจสอบย้อนได้

12. การติดตั้งและการใช้งาน

12.1 การติดตั้งครั้งแรก (First-time Setup)

การติดตั้งระบบ LMDS เป็นครั้งแรก ประกอบด้วยขั้นตอนหลัก 5 ขั้นตอน ซึ่งต้องดำเนินการตามลำดับ เพื่อให้ระบบพร้อมใช้งานอย่างสมบูรณ์

ตารางที่ 5: ขั้นตอนการติดตั้งครั้งแรก

ขั้นตอน	รายละเอียด
---------	------------

ขั้นตอน	รายละเอียด
1	ผูก Apps Script กับ Google Spreadsheet ที่ต้องการใช้งาน
2	ตั้งค่า API Key จากเมนูระบบ เพื่อเชื่อมต่อกับ SCG API และ Google Maps API
3	รันคำสั่งสร้างซีตทั้งหมดตาม SCHEMA ที่กำหนดไว้ใน <code>02_Schema.gs</code>
4	เติมข้อมูล <code>SYS_TH_GEO</code> และรันคำสั่งเตรียมดิกชันนารีสำหรับการ Normalize ที่อยู่
5	ทดสอบดึงข้อมูลตัวอย่างและรัน Pipeline 1 รอบ เพื่อยืนยันว่าระบบทำงานถูกต้อง

12.2 การใช้งานหลัก

เมื่อระบบติดตั้งเรียบร้อยแล้ว การใช้งานหลักประกอบด้วย 3 กิจกรรมหลัก ที่ทีมปฏิบัติการต้องดำเนินการอย่างสม่ำเสมอเพื่อให้ข้อมูลในระบบมีความถูกต้องและทันสมัยอยู่เสมอ

- **ดึงงานรายวัน:** ดึงข้อมูลงานรายวันจาก SCG API ลง `ตารางงานประจำวัน` เพื่อเริ่มกระบวนการประมวลผล
- **รัน Full Pipeline:** รัน Pipeline เติมรูปแบบเพื่อประมวลผลข้อมูลเข้าฐาน Master + `FACT_DELIVERY` ระบบจะทำการ Normalize, Resolve, และจับคู่อัตโนมัติ
- **ตรวจ Q_REVIEW:** ตรวจสอบคิว `Q_REVIEW` เพื่อทำ Human-in-the-loop สำหรับเคสคำถามที่ระบบไม่สามารถตัดสินใจได้ด้วยตนเอง โดยเลือก Action ที่เหมาะสม (CREATE_NEW / MERGE / IGNORE / ESCALATE)

12.3 หมายเหตุสำคัญก่อน Production Run

ข้อควรระวังก่อนรัน Production

- ควรยืนยันว่า Header ทุกซีตตรงกับ `SCHEMA` ที่กำหนดไว้ใน `02_Schema.gs` หากไม่ตรง ระบบอาจบันทึกข้อมูลผิดคอลัมน์
- ควรตรวจว่า `M_ALIAS` ถูกสร้างแล้วและเรียงคอลัมน์ถูกต้อง เพื่อให้ Hybrid Alias Architecture ทำงานได้อย่างสมบูรณ์
- ระบบรองรับการรันกับข้อมูลใหม่ได้ทันทีตามแนวทางที่ตกลงกัน โดยไม่จำเป็นต้อง Backfill ข้อมูลเก่า

13. หมายเหตุการใช้งานจริง (Production Notes)

ข้อควรทราบสำหรับการใช้งานระบบ LMDS ในสภาพแวดล้อมจริง (Production) มีดังต่อไปนี้ เพื่อให้การดำเนินงานเป็นไปอย่างราบรื่นและไม่เกิดปัญหาที่อาจส่งผลกระทบต่อข้อมูลหรือการทำงานของระบบ

- **ระบบพร้อมรับข้อมูลใหม่แบบเต็มระบบ:** ระบบสามารถรับและประมวลผลข้อมูลใหม่ได้ทันที โดยไม่จำเป็นต้องมีการเตรียมการพิเศษใดๆ สำหรับข้อมูลที่เพิ่งเข้ามา
- **ไม่บังคับ Migration/Backfill ข้อมูลเก่า:** ตามนโยบายรอบนี้ ระบบไม่บังคับให้ต้องย้ายหรือเติมข้อมูลเก่าย้อนหลัง ทำให้สามารถเริ่มใช้งานได้ทันทีโดยไม่กระทบข้อมูลที่มีอยู่เดิม
- **รักษา Header Order:** ต้องรักษาลำดับ Header ให้ตรงกับ Schema เสมอ การเปลี่ยนแปลงลำดับคอลัมน์อาจทำให้ระบบบันทึกข้อมูลผิดพลาดได้
- **อัปเดต Config + Schema พร้อมกัน:** ทุกการเปลี่ยนแปลง Schema ควรอัปเดต `01_Config.gs` และ `02_Schema.gs` พร้อมกันเสมอ เพื่อให้การตั้งค่าและโครงสร้างข้อมูลสอดคล้องกัน